

Desafio Full Stack: Marketplace Multi-ONG

Contexto

Queremos criar um marketplace para conectar consumidores com ONGs parceiras. Cada ONG mantém seu estoque de produtos (artesanato, alimentos, vestuário etc.) e os disponibiliza para venda em um portal público. O objetivo é fornecer uma plataforma robusta e funcional.

Estrutura do Desafio

O desafio está dividido em duas etapas. Todos os candidatos devem entregar a Etapa 1. A Etapa 2 é focada em arquitetura avançada, resiliência e otimização, sendo esperada para candidatos que buscam posições de maior senioridade.

Etapa 1: O MVP Funcional (Obrigatório)

1. Área da ONG (Restrita)

- **CRUD de Produtos:** Cadastro/edição de name, description, price (suportar decimais), category, image_url, stock_qty e weight_grams (peso em gramas).
- **Segurança Multi-Tenancy (Crítico):** Implementação rigorosa de tenancy por organization_id. O backend deve garantir que uma ONG **nunca** possa visualizar ou editar dados de outra. O organization_id deve ser derivado do usuário autenticado (token/sessão), nunca confiado do client-side.
- **Autenticação:** Implementação de um mecanismo de login funcional.

2. Portal Público

- **Catálogo de Produtos:** Lista paginada de produtos de todas as ONGs.
- **Filtros Manuais:** Filtragem por categoria e faixa de preço.
- **Busca Inteligente (AI):**
 - O usuário digita em linguagem natural (ex.: "doces até 50 reais").
 - O sistema utiliza uma API de LLM para converter a entrada em filtros estruturados (ex: category, price_max, keywords).
 - Exibir ao usuário a interpretação aplicada (ex.: "Resultados para: Categoria=Doces; Preço ≤ 50").
- **Resiliência/Fallback (Crítico):** Se a chamada à AI falhar ou demorar muito (definir um timeout razoável), a busca deve reverter para uma busca simples por texto no nome e descrição.

3. Carrinho e Pedido

- Usuário seleciona itens e quantidades.

- Botão “Realizar Pedido” que persiste um registro estruturado do pedido no banco de dados.
- Estrutura mínima sugerida: Pedido (dados gerais) e Itens do Pedido (itens associados, incluindo preço no momento da compra e organization_id).
- *Nota: Pagamento real e baixa de estoque imediata não são necessários nesta etapa.*

4. Logs e Observabilidade Básica

- Registro de logs estruturados (ex: JSON) para as requisições: timestamp, rota, método, status, latência, e identificadores relevantes (userId, organization_id).
- Log específico para a Busca Inteligente: texto de entrada, filtros gerados, sucesso da AI (boolean), fallback aplicado (boolean).

Etapa 2: Complexidade e Arquitetura (Diferencial Sênior)

Esta etapa foca em desafios arquiteturais, consistência de dados e otimização. Complete os itens 5 e 6, e escolha UMA opção do item 7.

5. Consistência de Estoque e Concorrência (Obrigatório)

Evolua o processo de 'Realizar Pedido' para gerenciar o estoque de forma segura e imediata.

- **Validação e Baixa Atômica:** O processo de criação do pedido deve agora validar e reservar o estoque atômicamente durante a requisição.
- **Controle de Concorrência (Crítico):** Utilize Transações de banco de dados e mecanismos de bloqueio apropriados para garantir que o estoque nunca fique negativo e prevenir a sobre venda (overselling), mesmo sob alta concorrência.
- **Feedback Imediato:** Se algum item não tiver estoque suficiente, a transação deve falhar (rollback) e o usuário deve ser informado imediatamente.

6. Processamento Assíncrono e Resiliência (Obrigatório)

Tarefas subsequentes à confirmação do pedido não devem bloquear a resposta ao usuário.

- **Arquitetura Assíncrona:** Introduza um mecanismo para processamento assíncrono (ex: filas e workers).
- **Pós-processamento:** Após o pedido ser criado e o estoque baixado com sucesso, tarefas assíncronas devem ser disparadas:
 1. Simulação de Pagamento: Simular chamada a um gateway externo (com um delay e chance de falha).
 2. Notificação: Simular o envio de uma notificação para a ONG e o cliente (logar a ação é suficiente).
- **Idempotência e Retentativas:** O sistema deve ser capaz de re-tentar tarefas em caso de falha transitória. As operações devem ser idempotentes para evitar efeitos colaterais duplicados se a mesma tarefa for processada mais de uma vez.

7. Aprofundamento Técnico (Escolha UMA opção)

Escolha e implemente apenas UMA das opções abaixo para demonstrar profundidade técnica:

- **Opção A: Busca Avançada (Performance e Relevância)**

Assumindo um catálogo grande, implemente uma busca mais eficiente que o padrão (ex: ILIKE). Utilize recursos avançados do seu banco de dados (ex: Full-Text Search ou Busca Vetorial/Semântica) para a busca por palavras-chave e o fallback da AI.

- **Opção B: Otimização e Caching (Escalabilidade)**

Implemente uma estratégia de Caching para otimizar a performance do portal público. Utilize um cache distribuído (ex.: Redis). Cachear listagens de produtos frequentes e descrever/implementar a estratégia de invalidação quando um produto é atualizado.

- **Opção C: Complexidade de Negócio (Logística Multi-Origem)**

Pedidos podem conter itens de diferentes ONGs. Implemente o cálculo de frete simulado *por ONG*, utilizando o campo `weight_grams` (ex: Frete Fixo + custo por peso). Atualize a estrutura do pedido para armazenar o custo de frete por organização e exiba o pedido agrupado por ONG na interface.

Requisitos Técnicos Gerais

- **Stack:** Qualquer stack moderna (Backend e Frontend) é aceitável (ex: Node.js, Python, Go, React, Vue).
- **Banco de Dados:** Banco de dados relacional. PostgreSQL é fortemente recomendado, especialmente para a Etapa 2.
- **Infraestrutura (Obrigatório):** O setup local de todos os serviços (DB, API, Frontend, e serviços adicionais como Redis ou Filas, se aplicável) deve ser feito via Docker/Docker Compose.
- **Testes:** A presença de testes automatizados (unitários/integração), especialmente nas áreas críticas (ex: criação de pedido, segurança), será considerada um diferencial.

Entregáveis (Formato da Entrega)

Repositório público no GitHub contendo:

1. Código do front e back.
2. Scripts de migração do banco e seed de exemplo (mínimo 2 ONGs, 5 produtos cada).
3. `/.env.example` (com placeholders, sem credenciais reais).
4. **README.md detalhado:**
 - Passo a passo completo para rodar localmente (usando Docker Compose).
 - Esquema do banco de dados (ERD ou descrição textual).
 - Descrição das principais rotas da API (públicas e restritas).
 - **Busca AI:** Como configurar (variáveis de ambiente), tempo limite adotado e descrição do mecanismo de fallback.
 - **Logs:** Como visualizar e o formato dos registros.
 - **Para Etapa 2 (se aplicável):**
 - Diagrama simples da arquitetura.
 - Explicação de como a concorrência de estoque foi tratada (Item 5).
 - Descrição do fluxo assíncrono e como a idempotência foi garantida (Item 6).
 - Detalhes da implementação da opção escolhida (Item 7).
 - Decisões de design, trade-offs, limitações conhecidas e próximos passos.

Prazo

Concluir em 1 semana a partir do recebimento e enviar o link do repositório GitHub.